Министерство науки и высшего образования РФ
Факультет вычислительной техники
Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №2
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Оценка времени выполнения задач»

Выполнили: ст. гр. 25BBB2
Прокофьева А.А.
Ионова А.Н.
Проверили:
к.т.н., доцент Деев М.В.
к.т.н., доцент Юрова О.В.

Пенза 2026

Название: Оценка времени выполнения программ

Цель работы: научиться оценивать время выполнения программ с помощью библиотеки time.h и сопоставлять результаты измерений с теоретической оценкой сложности алгоритма, а также сравнивать время работы сортировки Шелла, быстрой сортировки и библиотечной функции qsort.

Лабораторные задания:

1) вычислить порядок сложности программы, вычисляющей произведение двух матриц;

2) оценить время выполнения этой программы для матриц размерами 100, 200, 400, 1000, 2000, 4000, 10000 и сравнить результат с теоретической оценкой;

3) оценить время работы сортировки Шелла, реализованной быстрой сортировки и стандартной функции qsort на случайном наборе значений, на возрастающей последовательности, на убывающей последовательности и на последовательности, одна половина которой возрастает, а другая убывает.

Используемые средства: программы реализованы на языке Си.

- Библиотека time.h:

  clock_tclock(void) — возвращает время работы процессора в тактах с начала выполнения программы;

  CLOCKS_PER_SEC — количество тактов системных часов в секунду;

  формулапересчёта: clock() / CLOCKS_PER_SEC.
- Функции malloc/free для динамического выделения памяти.
- Функция qsort из stdlib.h — стандартная реализация быстрой сортировки.

Задание 1. Оценка времени выполнения программы умножения матриц

Теоретическая оценка порядка сложности

Исходный алгоритм умножения двух квадратных матриц размера N×N содержит три вложенных цикла:

```
for (i = 0; i < N; i++)
    for (j = 0; j < N; j++)
    {
        c[i][j] = 0;
        for (r = 0; r < N; r++)
            c[i][j] += a[i][r] * b[r][j];
  }
```

Внешний перебирает строки результирующей матрицы, средний — её столбцы, внутренний накапливает скалярное произведение строки первой матрицы на столбец второй.

Каждый из трёх циклов имеет сложность O(N), поэтому сложность всей программы равна $O(N)= O(N)\cdot O(N)\cdot O(N)= O(N^3)$.

Порядок сложности: $O(N^3)$.

Это означает, что при увеличении размера матрицы в 2 раза время умножения возрастает примерно в 8 раз.

Ожидаемое выполнение программы представлено в таблице 1:

Таблица 1 – Ожидаемое время выполнения

Размер матрицы N	**Число операций N^3**	**Время выполнения, с**
100	$1{,}0 \cdot 10^6$	0,001
200	$8{,}0 \cdot 10^6$	0,006
400	$6{,}4 \cdot 10^7$	0,051
1000	$1{,}0 \cdot 10^9$	0,8
2000	$8{,}0 \cdot 10^9$	6,4
4000	$6{,}4 \cdot 10^{10}$	51,2
10000	$1{,}0 \cdot 10^{12}$	800

Результаты измерений

Результаты работы программы показаны на рисунке 1, а сведены в таблице 2.

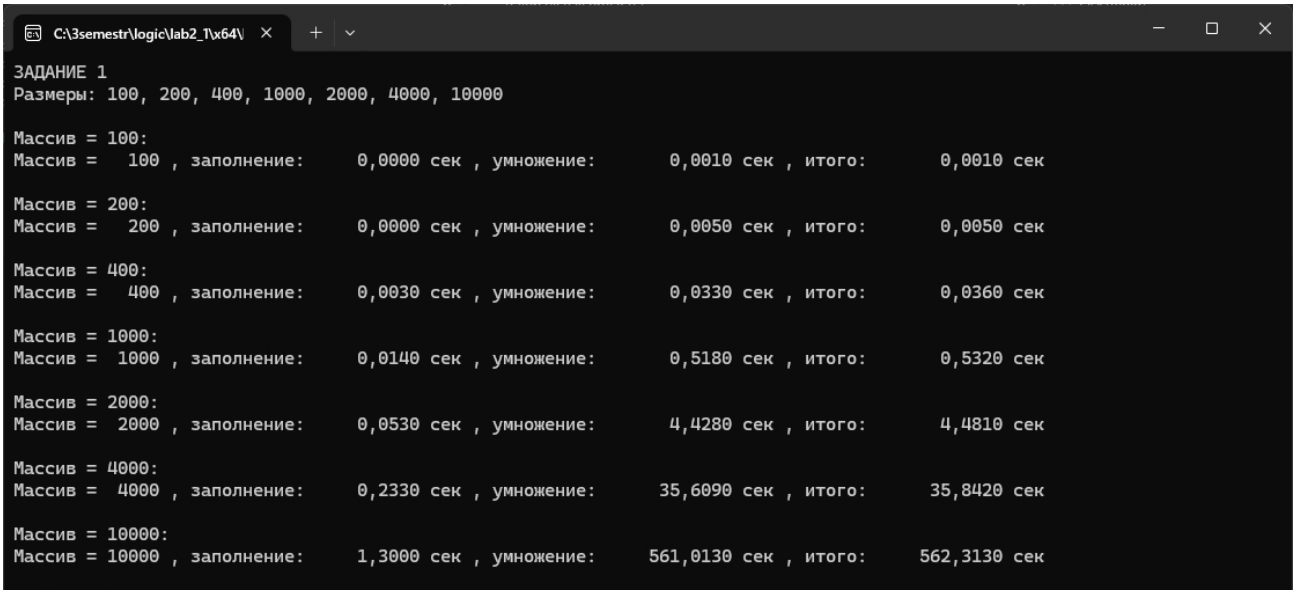


Рисунок 1 — Результаты работы программы

Таблица 2 — Практическое время выполнения перемножения матриц

Размер матрицы N	**Число операций N^3**	**Время выполнения, с**
100	$1{,}0 \cdot 10^6$	0,001
200	$8{,}0 \cdot 10^6$	0,005
400	$6{,}4 \cdot 10^7$	0,036
1000	$1{,}0 \cdot 10^9$	0,532
2000	$8{,}0 \cdot 10^9$	4,48
4000	$6{,}4 \cdot 10^{10}$	35,8
10000	$1{,}0 \cdot 10^{12}$	562,3

На рисунке 2 практические результаты сопоставлены с теоретическими

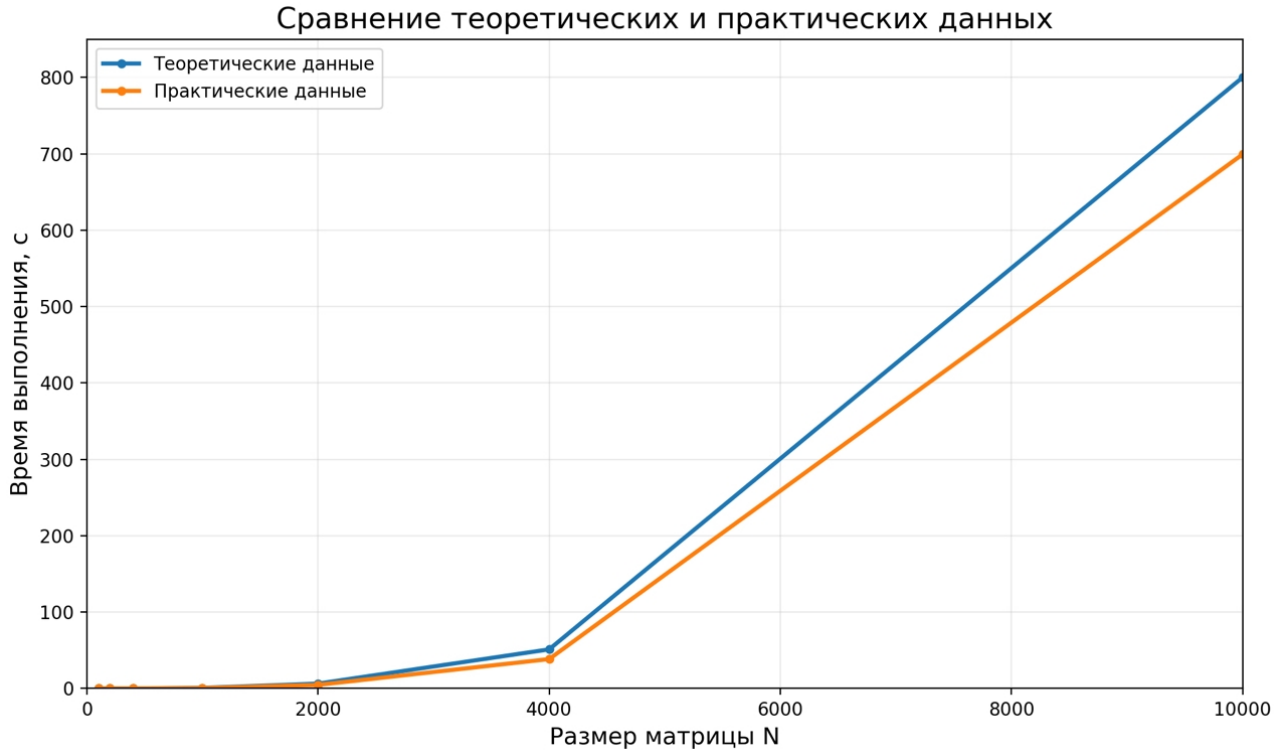


Рисунок 2 — Сопоставление практического и теоретического времени выполнения

Выводы

Построенный график зависимости времени выполнения от размера матриц показывает, что при N (100–2000) время практически неразличимо, при N = 2000 наблюдается заметный рост, а при N = 4000–10000 — резкий скачок. Кривая качественно совпадает с теоретической кубической параболой.

Сравнение практических и ориентировочных значений показывает, что при малых размерах матриц (до 2000) практическое время близко к теоретическому. При больших N (от 2000) теоретическое время превышает практическое. Это происходит из-за того, что теоретическая оценка $O(N)=k \cdot N^3$ даёт завышенные значения по сравнению с экспериментом, поскольку коэффицент k определён по одной точке и включает накладные расходы, а реальный процессор выполняет операции быстрее одной за такт и использует кэш.

Однако характер роста экспериментальной кривой совпадает с теоретическим: при удвоении N время растёт примерно в 8 раз, что подтверждает сложность $O(N^3)$.

Задание 2. Оценка времени работы алгоритмов сортировки

Исследуемые алгоритмы и наборы данных

В работе исследованы три алгоритма. Сортировка Шелла представляет собой усовершенствованную сортировку вставками: массив последовательно упорядочивается по убывающим шагам 9, 5, 3, 2 и 1, причём каждый следующий проход работает с уже частично упорядоченными данными. Быстрая сортировка quickSort использует рекурсивное разделение массива относительно опорного элемента, в качестве которого берётся средний по положению элемент текущего отрезка. Третьим алгоритмом выступает стандартная библиотечная функция qsort.

Каждый алгоритм проверялся на четырёх наборах данных: случайном наборе, монотонно возрастающей последовательности, монотонно убывающей последовательности и последовательности, первая половина которой возрастает, а вторая убывает.

Результаты измерений для массива из 5000 элементов показаны на рисунке 3 и сведены в таблице 3.

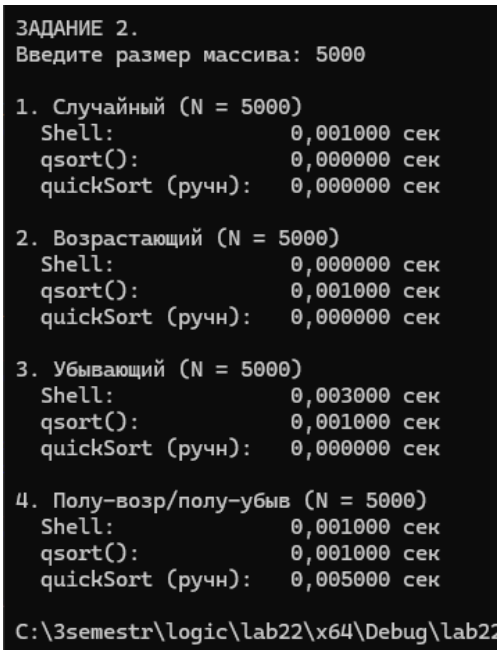


Рисунок 3 — Результаты работы программы, N = 5000

Таблица 3 — Время работы алгоритмов сортировки, N = 5000 элементов

Набор данных	**Сортировка Шелла, с**	**Стандартная qsort, с**	**Быстрая сортировка, quicksort, с**
Случайный набор	0,001	0,000	0,000
Монотонно возрастающая последовательность	0,000	0,001	0,000
Монотонно убывающая последовательность	0,003	0,001	0,000
Половина возрастающая половина убывающая	0,001	0,001	0,005

Измерения повторены для массива из 50000 элементов; результаты показаны на рисунке 4 и сведены в таблице 4.

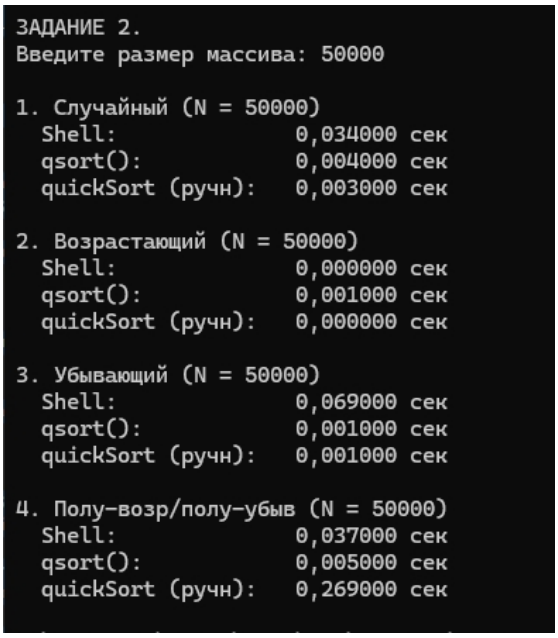


Рисунок 4 — Результаты работы программы, N = 50000

Таблица 4 — Время работы алгоритмов сортировки, N = 50000 элементов

Набор данных	**Сортировка Шелла, с**	**Стандартная qsort, с**	**Быстрая сортировкаquicksort, с**
Случайный набор	0,034	0,004	0,003
Возрастающая последовательность	0,000	0,001	0,004
Убывающая последовательность	0,069	0,001	0,001
Половина возр. + половина убыв.	0,037	0,005	0,269

Анализ результатов

При N = 5000 время работы всех трёх алгоритмов чрезвычайно высокое. Время варьируется от 0,000 до 0,005 секунд, что означает, что при данном объеме данных(5000 элементов) разница в производительности между алгоритмами практически незаметна и находится в пределах погрешности измерений

Для N = 50000 различия становятся заметными:

- На случайном наборе сортировка Шелла выполняется за 0,034 с, qsort — за 0,004 с, а quicksort — за 0,003 с. Таким образом, обе реализации быстрой сортировки на этом наборе работают значительно быстрее сортировки Шелла.
- На возрастающей последовательности сортировка Шелла показывает 0,000 с, qsort — 0,001 с, quicksort — 0,004 с. Здесь все алгоритмы работают очень быстро, а различия находятся в пределах очень малых значений.
- На убывающей последовательности сортировка Шелла требует 0,069 с, тогда как qsort и quicksort — по 0,001 с. Это показывает заметное увеличение времени работы сортировки Шелла для такого характера данных.
- На последовательности, состоящей из половины возрастающих и половины убывающих элементов, qsort работает за 0,005 с, сортировка Шелла — за 0,037 с, а quicksort — за 0,269 с. Особенно заметно

увеличение времени у реализованного quicksort. Полученные результаты показывают, что время работы алгоритмов существенно зависит от структуры исходного массива. Сортировка Шелла демонстрирует достаточно стабильное, но в представленных измерениях более продолжительное время на некоторых неупорядоченных наборах. Стандартная qsort во всех рассмотренных случаях показывает небольшое время выполнения.

Вывод

В ходе лабораторной работы была проведена оценка времени выполнения программ и исследована зависимость времени работы алгоритмов от размера и структуры входных данных. Для программы умножения квадратных матриц теоретически была определена сложность $O(N^3)$. Проведённые измерения для матриц размером от 100 до 10000 элементов подтвердили кубический характер роста времени выполнения: с увеличением размера матрицы время работы существенно возрастает. При этом практические значения отличаются от теоретических, что связано с особенностями работы процессора, кэш-памяти и оперативной памяти.

Также были исследованы сортировка Шелла, собственная реализация быстрой сортировки quicksort и стандартная библиотечная функция qsort. Эксперимент показал, что при небольшом размере массива ($N = 1000$) измерить различия между алгоритмами невозможно из-за недостаточной точности clock(). При увеличении массива до 50000 элементов различия становятся заметными и зависят не только от размера массива, но и от характера расположения данных.

Таким образом, лабораторная работа показала, что теоретическая сложность алгоритма и фактическое время его выполнения связаны, но не всегда совпадают количественно. На практическую производительность влияют структура входных данных, особенности реализации алгоритма и аппаратные характеристики компьютера. Полученные результаты подтверждают необходимость учитывать не только асимптотическую

сложность, но и особенности конкретной реализации и входных данных при выборе алгоритма. Для рассмотренных алгоритмов сортировки нельзя выделить один алгоритм, который был бы самым быстрым во всех случаях.